The Pollis Guide

How to use Pollis, one topic at a time

How this guide works

Each topic takes two facing pages: what the feature is for, where to find it, the steps, its limits, and what to do when something goes wrong. Read the topics in any order.

Menu paths are written **Compose > Collaboration > Join Session...**, and buttons as they appear, such as **Save As...**. A setting is named by its id, such as `pollis.collaboration.signalling`, which you can paste into the search box of **Settings**.

This guide describes Pollis as it is on 16 October 2026. What is planned but not available yet is said to be not available.

Getting Pollis

Pollis is free and open source, for Windows, macOS and Linux: github.com/saragga/pollis/releases/latest.
Questions and problems: **Help > Report Issue**, or contact@pollis.org.

Contents

2 Getting Started
4 Inside a Panel
6 Data and Databases
8 Customisation
10 Toolboxes
13 Replication, Methods and Workflows
14 Replication
16 Methods
18 Pollis Packages
20 Reports
22 RSS Feeds
24 Media Coverage
26 Alerts
28 Event Studies
32 Collaboration
34 Orchestration
36 Literature Radar
38 Snowball Search Map
40 Built-in Databases
42 Appendix: Reference Keys

© 2026 Antonio Saragga Seabra · Licensed under CC BY-SA 4.0 · Draft, 16 October 2026

Getting Started

Welcome page · Julia REPL

Installing Pollis and Julia, the first launch, and how the Julia packages a panel needs are installed.

Pollis is a single application with no account and no licence key. It writes Julia code; Julia, which runs that code, is installed separately, from the Welcome page, in a few minutes.

Install Pollis

Download the file for your system from github.com/saragga/pollis/releases/latest.

- **macOS** (Apple silicon), `Pollis-darwin-arm64.zip`:
  1. Unzip it and move `Pollis.app` to `Applications`.
  2. In Terminal, run `xattr -cr /Applications/Pollis.app`.
  3. Open Pollis.
- **Windows** (x64), `PollisSetup-x64.exe`:
  1. Double-click it. If SmartScreen warns you, choose **More info**, then **Run anyway**.
  2. Click **Install**. No administrator rights are needed.
  3. Pollis opens when the installation finishes; later, open it from the Start menu.
- **Linux** (x64), `Pollis-linux-x64.tar.gz`:
  1. Run `tar -xzf Pollis-linux-x64.tar.gz`.
  2. Run `Pollis-linux-x64/pollis`.

> **The apps are not signed yet**
>
> That is why macOS needs the `xattr` step and Windows may warn you. Download Pollis only from the link above.

Install Julia

If Julia is not installed, the Welcome page says so under **Status**: *Julia is not installed*.

1. Choose **Install Julia**, then **Download and Install**.
2. Pollis installs it with juliaup, the official Julia version manager, in a terminal. The terminal says when Juliaup and Julia have been installed.
3. The Welcome page's **Status** then says *Julia found*, and the Julia REPL starts.

If Julia is already installed somewhere Pollis cannot find it, choose **Set Julia Path** instead, and give the path of the Julia program in the setting `julia.executablePath`.

The first start of the Julia REPL

When Pollis opens, the Julia REPL starts in the panel at the bottom. The first time, it installs DuckDB and PollisDatasets, makes the four built-in databases, PollisDatasets, PollisRSS (empty until feeds are read), PollisJournals and PollisReferences (Built-in Databases, page 40), and says *This happens only once*. They go into an environment of Pollis's own, `~/.pollis/julia`, which comes after yours: your own versions of these packages are used first. At later starts, PollisDatasets, PollisJournals, PollisReferences and PollisRSS are updated from GitHub when there is a newer version; offline, the installed copies are kept.

The packages a panel needs

The **Powered by:** line at the top of a panel names its Julia packages. Beside it, a tick means they are all installed; **Install** means some are missing, and installs them in the Julia REPL, in your active Julia environment. You do not have to: the code a panel sends to the Julia REPL or to a notebook starts with a line such as

```
Pollis.packages("GLM", "Plots", "PollisDatasets")
```

which installs whichever of them Julia cannot load, and does nothing when all are there. Replications, notebooks and Code Examples begin the same way. The first run of a panel can therefore take a few minutes; the next ones do not.

Updates

Pollis asks GitHub once a day for a newer release. When there is one, a badge appears on the **Manage** gear, and **Download Update** opens the file for your system in your browser: install it as above. At the first start of a new version, Pollis offers **What's New**; **Help > What's New in Pollis** opens the same notes at any time. To stop the checks, set `update.mode` to `none`.

The Welcome page and notifications

Pollis tells you things in four places, each with its own purpose:

Status	Whether Pollis is ready: Julia and its version, and the built-in databases, PollisDatasets, PollisRSS, PollisJournals and PollisReferences. Always shown; when something is missing it is a warning with its button.
Announcements	New releases of Julia, DBInterface, PollisDatasets and PollisTeX, until you dismiss them. Shown only when there is one.
RSS	The newest unread items of the feeds you follow (page 22); a camera or a microphone marks a video or a podcast, and a journal's items have the tag of its level and metrics. Its **Alerts** tab shows your alerts and those that fired (page 26).
Notifications	The bell at the bottom right: what happens while you work, such as new datasets, new content or a toolbox to reload.

A faster start

If you do not want the Julia REPL at every start, clear `julia.startREPLOnStartup`. It then starts the first time you send code to it.

If something goes wrong

macOS: "Pollis is damaged and can't be opened"	Run `xattr -cr /Applications/Pollis.app` in Terminal, then open Pollis again.
Windows: "Windows protected your PC"	Choose **More info**, then **Run anyway**.
***Julia is not installed*, though it is**	Choose **Set Julia Path** and give the path of the Julia program.
The first run of a panel seems stuck	Look at the Julia REPL: packages are being installed and compiled. Wait for the prompt to come back.
Install stays after installing	The tick appears once Julia has finished; it checks again every few seconds.

Not available yet

A Mac build for Intel processors, and signed apps.

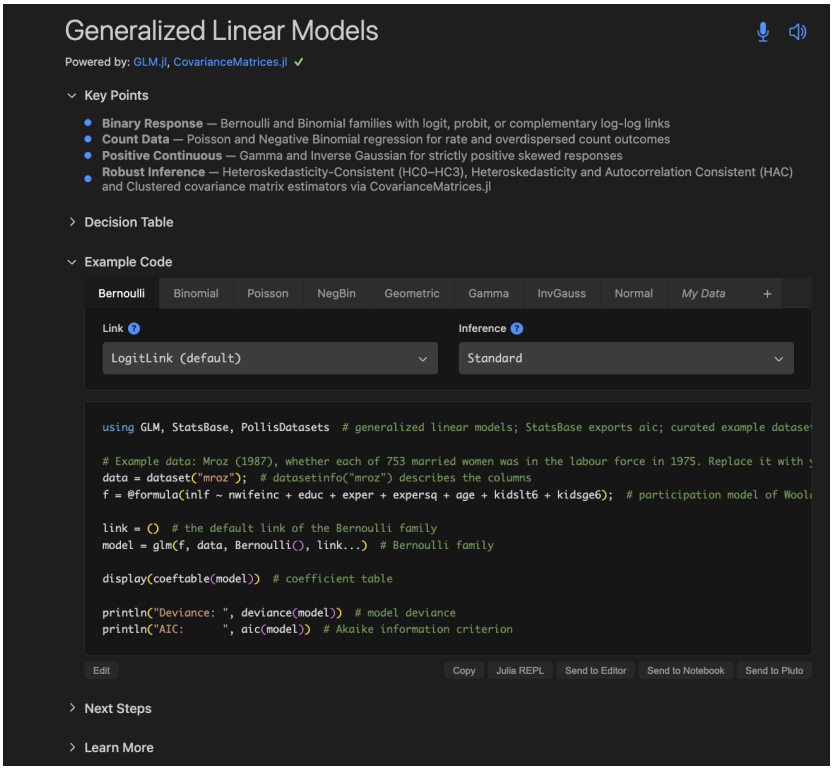
See also Inside a Panel, page 4. Data and Databases, page 6.

Inside a Panel

Explore · Model · Simulate · Optimise

What a panel is made of, and how to go from its example to code of your own.

Each item of the **Explore**, **Model**, **Simulate** and **Optimise** menus opens a *panel*: a page about one method, with an example you can run as it is, with simulated data or a dataset from PollisDatasets, and then make your own.



From top to bottom

- **The title**, and the **Powered by:** line: the Julia packages the panel uses, each a link to its documentation, with the tick or **Install** (see Getting Started, page 2).
- **Key Points**: what the method does and when to use it, in a few lines.
- **Decision Table**, in some panels: which variant to use for which data.
- **Illustration**, in some panels: a picture of the method or its variants.
- **Example Code**: the tabs of the panel's variants (above, **Bernoulli**, **Binomial**, …), its inputs, and the code. Changing an input changes the code at once.
- **Next Steps** and **Learn More**, at the bottom (next page).

Click a section's name to open or close it.

Run the example

The buttons below the code send it where you want it:

- **Julia REPL** runs it in the Julia REPL, starting it if needed.
- **Send to Notebook** opens it in a new Jupyter notebook, with the Julia kernel.
- **Send to Pluto** opens it in a new Pluto notebook, in Pollis's browser.
- **Send to Editor** opens it in a new, unsaved Julia file, and **Copy** copies it.

Code sent to the Julia REPL or to a notebook starts with the line that installs any missing package.

Make the example your own

1. Choose **Edit** below the code, and change it: your data, your variables, your options.
2. Choose **Save**, or **Cancel** to leave it as it was. A saved example is marked **Customised**, and the panel shows your version from then on.
3. To go back to the original, choose **Restore Default**, then **Confirm Restore** within four seconds. Your version goes to the Trash.

Add a tab of your own

1. Choose **+** at the end of the tabs, type a name and press **Enter**.
2. The new tab, marked **Added**, opens empty, for editing: write or paste your code, and **Save**.
3. To remove it, choose the **×** on the tab, then **Delete**; it goes to the Trash.

Your examples and tabs are files in `~/.pollis/examples`, one folder per panel. They are kept when Pollis is updated.

Next Steps

Visualise, Diagnose, Predict, Compare, Interpret: examples that carry on from the Example Code, using its results. Next Steps opens once you have run or sent the Example Code of the tab (the **?** beside it says so). Its examples have the same buttons, and can be edited and restored in the same way.

Learn More

- **Local Wikis**: a reference page on the method, in Pollis.
- **Notebook Tutorials**: notebooks that work through the method.
- **Multimedia Tutorials**: videos and talks. **Add / Edit Video** adds your own.
- **Explore References**: the papers and books behind the method, with **Open in Browser** and **Copy BibTeX to Clipboard**. **Add / Edit Reference** adds your own, from BibTeX, a link or a DOI; they go into your references, in PollisReferences (page 40), and **Edit** corrects a work wherever it is cited.

> **The example data are an example**
>
> They show how the method works. To use it on your own data, change the lines that load the data, and check that the variables and options still suit your question.

If something goes wrong

Next Steps is dimmed	Run or send the Example Code of this tab first.
An error that a package is not found	Use **Julia REPL** or **Send to Notebook**, which install it, or **Install** on the **Powered by:** line.
The send buttons are greyed out	Finish editing (**Save** or **Cancel**); an added tab needs some code first.

> **Not available yet**
>
> The microphone and speaker buttons beside the title: voice comes in a later version.

See also Getting Started, page 2. Data and Databases, page 6. Customisation, page 8.

Data and Databases

Databases view · Explore > Access Files and Databases

The datasets that come with Pollis, your own databases, and the Databases view.

Pollis comes with a collection of real datasets, PollisDatasets, that its examples use, and connects to DuckDB, SQLite and PostgreSQL databases. All the work with data is done in Julia: the Databases view shows what is there and writes the code to reach it.

PollisDatasets

57 real datasets, such as `mroz`, `airpassengers`, `iris`, and `challenger`, each with its source and licence. In Julia:

- `datasets()` lists them;
- `datasetinfo("mroz")` describes one and its columns;
- `data = dataset("mroz")` loads one as a Tables.jl table.

They are installed the first time the Julia REPL starts (Getting Started, page 2). When new datasets are added, a notification says so.

The built-in database

The same datasets are also a DuckDB database, *PollisDatasets*, one table per dataset, named after it, with a description of the table and of each column. It is read-only, and is rebuilt only when PollisDatasets changes. The Welcome page's **Status** says when it is ready. Like the other built-in databases (page 40), it is a DuckDB file, not in the Lance format: Pollis also brings DuckDB's Lance extension, for Lance datasets such as tables of embeddings, but the built-in databases do not need it.

1. Open the **Databases** view, in the secondary side bar on the right (or choose **Open Databases View** in the notification of new datasets).
2. Click **PollisDatasets**, marked **built-in**. It connects in the Julia REPL as `pollis`.
3. Expand it to see its tables and their columns.

Then query it in the Julia REPL with SQL:

```
DBInterface.execute(pollis, "SELECT * FROM mroz WHERE educ > 12")
```

The Databases view

- Click a connection to connect it in the Julia REPL; **Disconnect** closes it.
- On a table, **Open in Table Viewer** shows its rows; right-click it for **Preview Rows in Julia REPL**.
- **Show Tables** and **Hide Tables**, beside a connection, expand or collapse it; **Refresh Schemas**, at the top of the view, shows tables you have just created.
- Right-click a connection for **Copy Name**, **Copy Connection Code**, **Edit Connection** and **Delete Connection**, which removes only the saved connection, never the database.

Three ways to use SQL

SQL should be at hand wherever you are working, without a second database program: all three use the same DuckDB engine, with the Lance extension, and attach the four built-in databases, read-only.

- **The** `duckdb>` **terminal**, for SQL alone, without starting Julia: **New DuckDB Terminal**, at the top of the Databases view (or **Databases: New DuckDB Terminal** in the Command Palette). It opens an empty database in memory; `.help` lists its commands, such as `.tables`, and Ctrl+C stops a long query.
- **The** `sql>` **mode** of the Julia REPL, for a query in the middle of Julia work: type > on an empty line, then the query, ending in `;`. The result is a Tables.jl table in `ans`, ready for Julia; Backspace on an empty line goes back to `julia>`.
- `db = PollisDB.connect()`, for Julia code and scripts: a DuckDB connection in memory, shown in the Databases view; `PollisDB.connect("sales.duckdb")` opens a file instead. Lance is loaded in the background, so the connection is ready at once.

Connect to your own database

1. In the Databases view, choose **New**, then **DuckDB**, **PostgreSQL** or **SQLite**.
2. Give a **Connection Name** and the **Julia Variable** that will hold the connection, and choose whether to keep it for all your work (**Global**) or for this folder only (**Current Workspace**).
3. For DuckDB or SQLite, choose the database file (**Choose File**). A DuckDB connection with no file is an empty database in memory: it can read CSV and Parquet files in place, and is lost when you disconnect. **Read Only** protects a file from changes.
4. For PostgreSQL, give the **Host**, **Port**, **Database** and **Username**.
5. Check the **Generated Code**, then **Save Connection**, or **Connect** to save it and connect.

A connection opened by your own code can be shown in the view as well:

```
con = PollisDB.show(DBInterface.connect(DuckDB.DB, "sales.duckdb"); name = "Sales")
```

> **PostgreSQL passwords**
>
> With **Ask for Password in the REPL** (the default), you type the password in the Julia REPL when you connect: it is never written in the code, in the REPL history or in a file. Without it, the password comes from `PGPASSWORD` or `~/.pgpass`.

Panels for data

- **Explore > Access Files and Databases**: **SQL Databases** (query, import a CSV, Parquet or JSON file, save a table, export a query), **CSV Files** and **Excel Workbooks**.
- **Explore > Access Data Libraries**: Hugging Face and Kaggle datasets, Yahoo Finance, Alpha Vantage, US SEC EDGAR, FRED and the European Central Bank. A provider that needs an API key shows **Set API key** on the **Powered by:** line; the key is kept in your system's secure storage.

If something goes wrong

The view shows no tables	Choose **Refresh Schemas**. A new database stays empty until you create a table in the Julia REPL.
PollisDatasets does not connect	The Julia REPL has not started yet, or is still installing: wait for the Welcome page to say the database is ready.
Cannot write to `pollis`	The built-in database is read-only: save your tables in a database of your own.
PostgreSQL does not connect	Check the host, port and **SSL Mode**; the **Connection Timeout** is 10 seconds.

> **Not available yet**
>
> International Monetary Fund Datasets, World Bank Indicators and the Kenneth French Data Library are in **Access Data Libraries**, greyed out.

See also Getting Started, page 2. Collaboration, page 32, for sharing datasets and databases.

Customisation

Pollis Views · Settings

Showing only the menus you need, and making Pollis yours.

Pollis has hundreds of panels. A *View* is a named set of the menu items Pollis shows: a course can keep only its own topics, a researcher only their field. The panels you hide are still there; another View shows them again.

Choose a View

The name of the View in use is in the title bar, before the layout buttons: **View: Max**. Click it for the Views menu, with a tick on the one in use. Two Views are built in:

- **Max** shows every menu item. It is the default.
- **Min** shows a short selection: the common plots and Descriptive Statistics in **Explore**, and a few models, simulations and optimisation panels.

Switching applies at once, to every menu. The built-in Views cannot be changed or deleted.

Make a View of your own

1. In the Views menu, choose **Create View...**, type its name, and choose the View it starts from: **Max** starts with every menu item. Or choose **Save View As...** to copy the View in use.
2. The new View becomes the one in use, and the **Views** panel opens in the secondary side bar, on the right, after **Databases**: the menus, with every item below them.
3. Untick an item to hide it, tick it to show it. Unticking a submenu hides everything in it; a submenu shows how many of its items are shown, such as **4/12**. Changes apply at once.

Explore, **Model**, **Simulate**, **Optimise**, **Extend** and **Compose** can be trimmed to the items you tick; in the other menus, such as **File** or **Help**, you can hide single items.

Edit, delete and share Views

- **Edit View...** makes one of your Views the one in use, and shows its items in the **Views** panel.
- **Delete View...** moves its file to the Trash, so it can be restored. Deleting the View in use switches to **Max**.
- **Open Views Folder** opens `~/.pollis/views`: each of your Views is a JSON file there. Copy a file to another computer, or give it to your students, to use the same View.

The View in use is kept in the setting `pollis.view`.

Settings

Choose the **Manage** gear at the bottom left, then **Settings**, and type in the search box a word or a setting's id, such as `julia.startREPLOnStartup`. **Themes > Color Theme**, in the same gear, changes the colours, light or dark.

Extensions

The Extensions pane, in the activity bar on the left, installs extensions from Open VSX, the open registry of extensions, as well as Pollis toolboxes (next topic). Some extensions published only by Microsoft are not on Open VSX, so they cannot be installed in Pollis.

What Pollis keeps for you

Your own things are files in `~/.pollis`, kept when Pollis is updated:

`views`	your Views
`examples`	your Example Code and added tabs (Inside a Panel, page 4)
`workflows`	your copies of the replications (page 14)
`methods, packages`	your new methods and their packages (page 16)
`practice`	the exercises you write (the Learning and Practice toolbox)
`rss`	the items of your RSS feeds (page 22)

> **A hidden item is only hidden**
>
> A View changes the menus, nothing else: the Command Palette and links between panels still reach every panel. A View is not a way to stop students from opening a panel.

If something goes wrong

A menu item has gone	The View in use hides it: switch to **Max**, or tick it in your View.
The items cannot be ticked	The View in use is built in: choose **Save View As...** to make an editable copy.
An extension is not found	It is not on Open VSX. Look for an open alternative there.

See also Toolboxes, page 10. Inside a Panel, page 4.

Toolboxes

Extensions pane > Toolboxes · Compose > New Panels and Toolboxes

Adding a whole field to Pollis with a toolbox extension.

A toolbox is an extension that adds a set of panels to the menus, with their wikis and notebook tutorials. It holds no program code: its panels are made like Pollis's own. Four are published:

- **Financial Econometrics**: ARIMAX, VAR and VECM, GARCH-type and regime switching models, forecasting, Bayesian asset-pricing factor models.
- **Economics**: business cycle filters, stochastic volatility, DSGE and heterogeneous-agent models, continuous-time macro-finance, game theory.
- **Mathematical Core**: symbolic maths, linear algebra, nonlinear systems, numerical differentiation and integration, ordinary and partial differential equations.
- **Learning and Practice**: multiple-choice exercises whose data and answers Julia computes, and a panel to write new ones; described in its own guide, *Learning and Practice*.

Install a toolbox

1. Open the Extensions pane, in the activity bar on the left. The **Toolboxes** section, at the top, lists the toolboxes you have not installed. Click a name for its details.
2. Choose **Install**. When Pollis asks, choose **Reload Now** or **Restart Extensions**.
3. The **Toolbox Panels** page opens: installing a toolbox does not change your menus until you choose its panels there (next page).

An installed toolbox moves to the **Installed** section. When a newer version is published, it comes back to **Toolboxes** with **Update**.

Choose its panels, and where they go

The **Toolbox Panels** page shows the toolbox on the left and **Your Menus (Preview)** on the right.

1. Tick the panels you want (**Select All**, **Select None**), or choose **Display Author's Suggestion** to see where the author would put them.
2. Click a blue line in the preview, **Put here**, to put the ticked items there. A ticked toolbox menu moves with its panels inside; untick it to move panels out of it.
3. Choose **Publish**. The page then says **In your menus as shown.**

A toolbox can go in **Explore**, **Model**, **Simulate**, **Optimise** or their submenus, or in **Extend**, a menu that appears only when a toolbox puts something in it.

To change them later, choose **Edit** on the toolbox in the **Installed** section, or **Toolbox Panels...** in the Command Palette (**Compose: Toolbox Panels...**). To remove a toolbox, choose **Uninstall** on it in the same section.

Make a toolbox

A teacher can package the panels of a course as one toolbox and give it to the students. This is done by your coding agent (Orchestration, page 34), with a skill that writes the toolbox for you.

1. Choose **Compose > New Panels and Toolboxes > Package a Toolbox...** The agent starts in a terminal.
2. Tell it which panels you want, by their menu titles, the menus they go in and their order, and the toolbox's name and description. It asks for what is missing.
3. It writes a short spec, builds the toolbox and its README, and says where the `.vsix` file is.
4. To install it, run **Extensions: Install from VSIX...** in the Command Palette and choose the file. Your students do the same with the file you give them. Then choose its panels, as on the previous page.

To change it, edit the spec, raise its version, build again and install the new file over the old one.

New panels

Compose > New Panels and Toolboxes > Create a Panel... has your coding agent write a new panel with you. **Create Panels from a Request...** writes several at once from a request file, which lists the toolbox and, for each panel, its topic, methods and menu. Without a request file open, it opens an example to fill in. New panels can then be packaged as a toolbox.

> **Toolboxes need the Internet to install**
>
> The list of toolboxes and their files come from GitHub. Once installed, a toolbox works offline.

If something goes wrong

"Could not load the list of toolboxes"	Check the Internet connection, then choose **Retry**.
Installed, but nothing in the menus	Choose its panels and **Publish**: **Edit** on the toolbox, in **Installed**.
The Toolbox Panels page is empty	Reload the window or restart the extensions, as Pollis asked, first.
A panel cannot go in a toolbox	Some Pollis panels have code of their own and are marked not portable; the agent says which.

See also Customisation, page 8. Orchestration, page 34.

Replication, Methods and Workflows

Replay a classic. Keep what it teaches.

Besides its replication record, a replicated article can leave two things behind: a new method, and a workflow.

1 Replication

It starts from an article, its input data, and either the results it publishes or its authors' code, in any language. If a method the article uses is missing from Julia's packages, it is written first. Then each result is printed beside the published one. Replication, page 14.

2 Methods

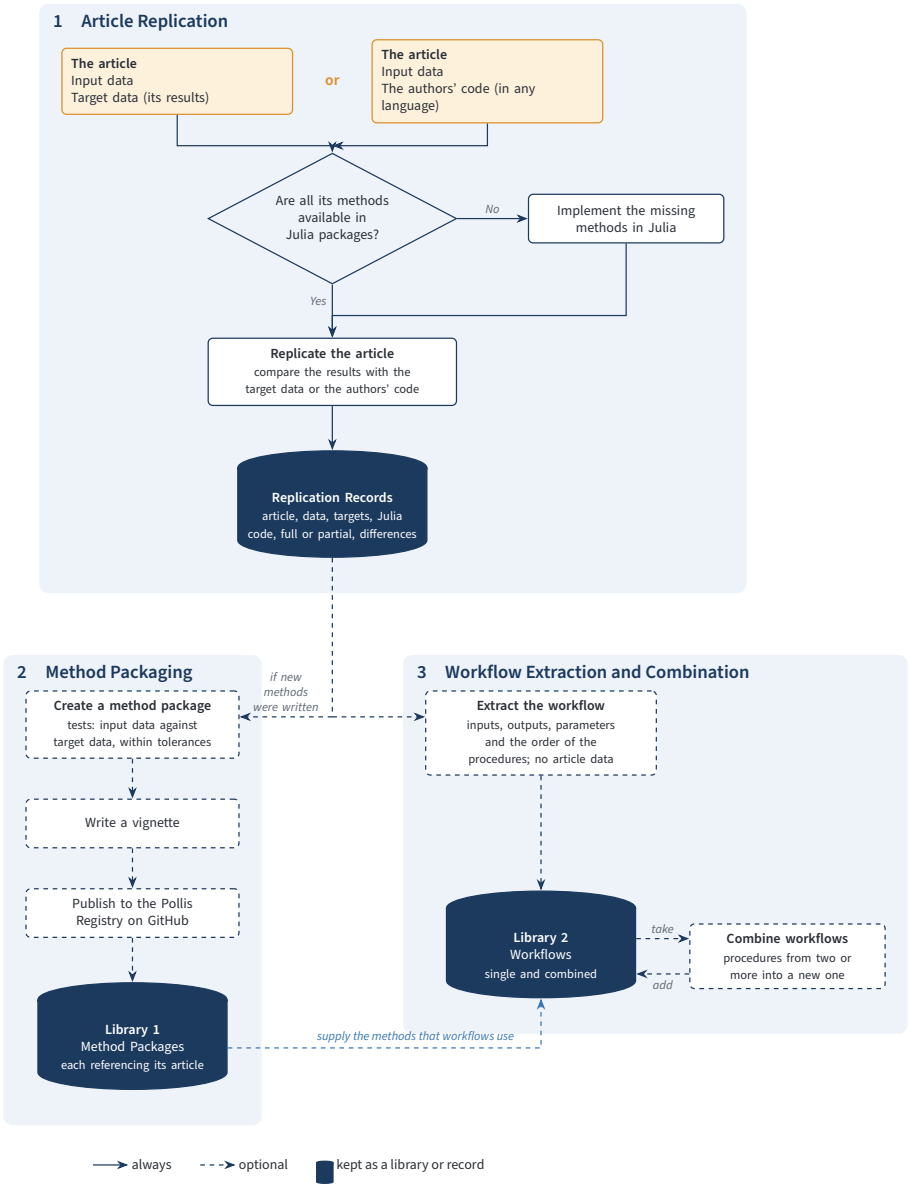
A method an article needed and no Julia package had becomes a documented Julia package. Its tests run the article's data and check the published results, within tolerances. It comes with a vignette, and can be published in the Pollis registry. Methods, page 16.

3 Workflows

A flat set of procedures, each with its inputs, its outputs and what it runs. Extracted from a replication, it runs on your own data; combined with others, its independent procedures run side by side.

Not available yet

Replication records (the article, its data, the targets, the Julia code, whether the replication is full or partial, and any differences); **Compose > Replication > Replicate a Classical Article...**; and **Browse Workflows...**, **Extract a Workflow...** and **Combine Workflows...** in **Compose > Workflows**.



Replication

Compose > Replication

Replaying a classic paper from its own data, step by step.

A *replication* reproduces a published analysis on its own public data: a Jupyter notebook that follows the article section by section and prints every published value beside Julia's. It shows a method at work on a real question, and that Julia gets the published answer.

The replications

Replication	Article
Panel data: is public capital productive?	Munnell (1990)
GARCH: volatility after non-trading days	Bollerslev and Ghysels (1996)
Survival: the Mayo model for primary biliary cirrhosis	Dickson et al. (1989)
Logistic regression: the Challenger O-rings	Dalal, Fowlkes and Hoadley (1989)
Threshold autoregression: the lynx and the sunspot cycles	Tong and Lim (1980)
Dynamic time warping: the UCR archive baselines	Dau et al. (2019)
Propensity score matching: the NSW experiment and its replications	Dehejia and Wahba (1999)
Random forests: Breiman (2001)	Breiman (2001)
LSTNet: long- and short-term patterns in multivariate time series	Lai, Chang, Yang and Liu (2018)

Open a replication

1. Choose **Compose > Replication > Browse Replications...**
2. The page lists the replications with their article (its DOI, or **Web Page** when it has none, and **Copy BibTeX**), the Julia packages and datasets they use, their size, any data they download, and the date they were last updated. Click **more** under a description to read it all.
3. Click a replication's name. The first time, Pollis makes your own copy in `~/.pollis/workflows`, and opens its notebook.

Your copy is yours to change: later openings show it as you left it.

Follow it

1. Choose the Julia kernel if the notebook asks for one.
2. Run the cells in order, from the top. The first code cell, `Pollis.packages(...)`, installs any package that is missing, so the first run takes longer.
3. Read each section beside the article: the text says which table, figure or page it replicates.

The data come from PollisDatasets or from the source the article used. The page says when a replication downloads data, and how much (**+ *size* download**); **download if asked** marks data that it downloads only when you agree.

Checking the results

Most comparisons are small tables: Julia's value in full, the same value rounded as the article prints it, the published value, and the number of decimals they share, or `DIFFERENT`. Each table's title says where the published value is printed, usually a page or table. Random forests averages 100 random runs, as Breiman did, so it asks instead whether Julia's average and Breiman's differ by less than two standard errors. All the notebooks but the panel-data one end with a section, *What differs, and why*.

New and updated replications

New replications, and new versions of the ones you have, can arrive between releases of Pollis: the page marks them **New** or **Updated** until you open them. When your copy is older than the new version, the page says *Your copy is older than this version* and offers **Open Updated Version**: the new notebook is saved beside yours, with its date in the name, and your copy is kept.

From a replication to a report

A replication's notebook can become a LaTeX report or slides: Reports, page 20. The GARCH replication has its own report template.

If something goes wrong

A cell fails: a package is not found	Run the first code cell, `Pollis.packages(...)`, then the others in order.
A download fails	Check the Internet connection; the source may be down for a while. Try again later.
You changed your copy and want the original	Rename or delete its folder in `~/.pollis/workflows`; opening the replication again makes a fresh copy.

Not available yet

Replicate a Classical Article... is in the menu but not built yet: it says so when chosen. So are **Browse Workflows...**, **Extract a Workflow...**, **Combine Workflows...**, **Create Operational Workflow...** and **Automations...**, in **Compose > Workflows**.

See also Reports, page 20. Methods, page 16. Data and Databases, page 6.

Methods

Compose > Methods

Turning a new method into a tested Julia package, with its vignette.

A *new method* is one no Julia package has yet, taken from an article: its data, the results the article publishes for them, and the code that reproduces them. Pollis makes it a Julia package whose tests are the published values, with a *vignette*, its documentation, and can publish it in the Pollis registry. Without data and expected results there is no new method.

Start a method

Choose **Compose > Under Construction...**. The method's page has five numbered steps, below; its title is at the top: click it to change it. Nothing is saved until you type or add something. Each method is a folder in `~/.pollis/methods`.

1. Article and Data

- **Article**: the **Article Link**. Pollis opens it in the integrated browser and keeps no copy; **Save a Copy...** saves one outside Pollis, by default in the `data` folder of your Pollis Workspace (Reports, page 20).
- **Input Tables**, the data the method is computed from: **Empty Table** to type one in, **Add from a Link**, **Add from PollisDatasets**, or **Add from Article**: give the numbers of the article's tables, such as `3.2, 5.8`, and click **Ask Agent**.
- **Target Tables**, the published results the method must reproduce: **Empty Table**, **Add from CSV Files**, or **Add from Article**. Every printed value becomes a test of the package.

A table typed in by the agent keeps every value as printed, and the agent lists the values it could not read for certain: check them against the article. The tests are only as right as the targets.

2. Write a Method

Write **Instructions** for the agent, saved in `INSTRUCTIONS.md`: what the method does, where it is in the article, what to watch for. Then click **Write Method**: your coding agent writes the code in `src`, with you, and runs the tests. **Edit method.toml** shows the method's own file: its tables, its call and its article. **Open Code** opens the code.

3. Write a Vignette

Once the method's tests pass, write notes on the vignette (`VIGNETTE.md`) and click **Write Vignette**: the agent writes it in LaTeX, with its PDF. **Edit Vignette** and **Open PDF** open them.

4. Prepare for Registration

Give the **Package Name**: a capital letter, then letters and digits, such as `ThresholdAR`; the Pollis registry asks for at least 5 characters. It also asks for a usage example and a docstring for every exported name: **Prepare for Registration** has the agent check the method and fix what it can.

5. Package and Publish

Each button runs in a terminal. **Package and Test** makes the package in `~/.pollis/packages`, runs its tests and checks the registry rules. **Package, Test and Push to GitHub** then pushes it to its own public repository, if its tests all pass. **Package, Test, Push and Register** then registers it, if it meets the rules. Pushing needs a **GitHub Owner**, a user or organization, and the GitHub CLI, `gh`, signed in; Pollis asks first: *Push to GitHub as …?*

Browse methods and read vignettes

Compose > Methods > Browse Methods lists yours, **In Progress** and **Published by You**, then the **Library** of nine methods by Pollis, from arc-fs to threshold autoregression: click one to open your own copy. **Delete** and **Delete Empty Methods** move folders to the Trash. **Help > Documentation > Package Vignettes** lists the vignettes of installed packages (they open offline) and, with the Internet, those **In the Registry**.

> **The agent steps cost tokens**
>
> Add from Article, Write Method, Write Vignette and Prepare for Registration run your coding agent, on your account.

If something goes wrong

…has no target tables, so nothing to test	Add the published results in **Target Tables**.
Copies are kept outside Pollis	Save the copy outside `~/.pollis`, e.g. in a Pollis Workspace.
The push fails	Sign in with `gh auth login` in a terminal, and check the GitHub Owner.

See also Replication, page 14. Orchestration, page 34.

Pollis Packages

github.com/Trumpingtons

The Julia packages written for Pollis, which you can also use without it.

Most panels use Julia packages from Julia's General registry. Fourteen packages are Pollis's own: nine methods, and five packages behind panels and features. All are public on GitHub, under Trumpingtons, with the MIT licence, and work in any Julia session, with or without Pollis. None is in the General registry.

The method packages

The nine methods of the **Library** in **Compose > Methods > Browse Methods** (Methods, page 16). Each is tested against the published results of the article it comes from, has a vignette, and is in the Pollis registry.

Package	Topic	Tested against
ArcFS	Boosting	Breiman's arc-fs, from his random forests article
BreslowCox	Survival analysis	The Mayo model of primary biliary cirrhosis
ErrorComponents	Panel econometrics	Baltagi's tables for the Grunfeld, gasoline and public capital data
ForestOOB	Random forests	Breiman's out-of-bag errors and variable importance
GLMExtras	GLM inference	Dalal, Fowlkes and Hoadley's Challenger O-ring analysis
LSTNet	Time series forecasting	Lai, Chang, Yang and Liu's LSTNet-Skip and their PyTorch code
PeriodicGARCH	Volatility	Bollerslev and Ghysels's periodic GARCH
PropensityMatching	K-nearest neighbours	Dehejia and Wahba, and its replications
ThresholdAR	Nonlinear dynamics	Tong and Lim's threshold autoregression

Install a method package

Add the Pollis registry once, then add packages by name, in the Julia REPL's package mode (type]):

```
pkg> registry add https://github.com/Trumpingtons/PollisRegistry
pkg> add ThresholdAR
```

Help > Documentation > Package Vignettes opens the vignettes of the packages you installed.

Packages behind panels and features

Package	What it does	Where Pollis uses it
PollisDatasets	57 small, well-known datasets for examples and teaching, as Tables.jl column tables	The built-in database (Data and Databases, page 6), **Add from PollisDatasets** in Methods, and the examples of panels and replications
PollisTeX	Typesets LaTeX from Julia with Tectonic, downloaded on first use	**Compose > Reports** (Reports, page 20)
EDGAR	Fetches SEC EDGAR filings and financial data	**Explore > Access Data Libraries > US SEC EDGAR**
HARE	Heteroskedasticity and autocorrelation estimators	**Model > Linear Models**: **Linear Models with Heteroskedasticity**, **with Autocorrelation** and **with Heteroskedasticity + AR(1)**
RangeVol	Range-based volatility estimators	The Range-Based Volatility Estimation panel, not in the menus at present

Install one

Pollis installs these packages by itself when a panel, a replication or a report needs them. To install one yourself, give its GitHub address, in the Julia REPL:

```
import Pkg; Pkg.add(url = "https://github.com/Trumpingtons/HARE.jl")
```

The datasets' own terms

The MIT licence covers PollisDatasets' code. Each dataset keeps the terms of its provider: `datasetinfo("penguins")` shows its description, the source to cite, its licence and its columns.

If something goes wrong

`add ThresholdAR`**: the package is not found** Add the Pollis registry first, as above.

`Pkg.add("HARE")`**: the package is not found** These five are in no registry: install with `Pkg.add(url = ...)`, as above.

See also Methods, page 16. Data and Databases, page 6. Reports, page 20.

Reports

Compose > Reports

From a notebook to a LaTeX report or slides.

Any Jupyter notebook, a replication's or your own, can become a LaTeX report (an article) or beamer slides, with its outputs and figures and a PDF beside it. Pollis does this in its Julia REPL, without an agent; a coding agent can then polish the draft into a finished document.

Before you start: a Pollis Workspace

Reports are written in the `reports` folder of a *Pollis Workspace*: an ordinary folder for your own work, outside Pollis. To make one, choose **File > New Pollis Workspace...**, then choose or create a folder, for example in your Documents, and click **Make Pollis Workspace**. Pollis puts a `pollis.toml` file and a `data` folder in it, and opens it. Next time, open that folder with **File > Open Folder...**.

Make a report or slides

1. Open the notebook, for example your copy of a replication (page 14).
2. Choose **Compose > Reports > Export Notebook as LaTeX...** for a report, or **Make Slides from a Notebook...** for slides. The notebook's own toolbar has the same two in its **Export** button: **LaTeX Report...** and **Beamer Slides...**. With no notebook open, Pollis asks you to choose one, starting in `~/.pollis/workflows`.
3. The Julia REPL installs PollisTeX if it is missing, runs the notebook's code cells in order and compiles the document.
4. The `.tex` opens in the editor, and the PDF beside it, in the integrated browser.

The files are in `reports/`*`NAME`* of the workspace, *NAME* being the notebook's: `report.tex` or `slides.tex` and its PDF, each cell's printed output in `outputs/cellN.txt`, and its figure in `figures/cellN.pdf`.

What the draft holds

- A *report* is a draft of the notebook's text cells, with the printed outputs and figures in their place and without the code.
- *Slides* have a frame for the text under each heading, and one for each output and each figure.
- A notebook with a *template*, a `latex` folder beside it, gets that document instead, with its own tables, figures and references. The GARCH replication ships one for its report.

Edit and compile

Edit the `.tex` as you like, then click **Compile LaTeX**, the play button at the top right of its editor: Pollis saves it, compiles it with PollisTeX in the Julia REPL and reloads the PDF. Exporting the notebook again refreshes the outputs and figures, and keeps your `.tex`: Pollis writes it only when the folder has none. To start the draft again, delete the `.tex` first.

Polish with Agent

1. Open the report's or the slides' `.tex`.
2. Choose **Compose > Reports > Polish with Agent...** The coding agent starts in a terminal in the report's folder.
3. It turns the draft into a finished document: sections, tables, captions and citations, from the numbers already computed. It does not run the notebook again, and it compiles the document as it goes: the PDF beside you reloads each time.

Polish runs on the Light tier of Model Routing (Orchestration, page 34).

> **Polish costs tokens**
>
> The menu item with the sparkle runs your coding agent, on your account. Read the result: an agent can still misplace a value, so check the numbers against the draft.

If something goes wrong

Reports are written in the reports folder of a Pollis Workspace	Click **New Pollis Workspace...**, or open your workspace folder.
A Pollis Workspace keeps your work outside Pollis	Choose a folder outside `~/.pollis`, for example in your Documents.
No PDF appears	A cell or the compile failed: read the error in the Julia REPL, fix it, and export or compile again.
Polish says to open the .tex	Open `report.tex` or `slides.tex` first, then choose **Polish with Agent...** again.

See also Replication, page 14. Orchestration, page 34.

RSS Feeds

Explore > RSS > RSS Feeds · Databases view > PollisRSS

Following news, journals, central banks, searches, podcasts and videos, and keeping what they bring.

Explore > RSS > RSS Feeds opens a page with the feeds you follow on the left, by topic, and their items on the right. Pollis reads every feed when it starts and then every hour; **Refresh** reads them at once. The feeds are read by your computer, not by a server of Pollis. At first, Pollis follows only the Julia news; a notification at the first launch points to the catalogue, where you choose the rest.

Every feed is RSS to Pollis, wherever it comes from: most come from the *catalogue*, about 520 feeds in 35 categories, from PollisRSS.jl and updated without a new Pollis, ranging from Classics and Philosophy to Economics and Astronomy, all listed on the page in alphabetical order; some are *searches* you save, at arXiv or PubMed; and some are *email alerts*, saved searches whose results arrive by email, which Pollis turns into feeds. They are read, shown, archived and added to Zotero alike.

Add a feed

Explore > RSS > Add Feed..., or **Add Feed** on the RSS Feeds page, opens a page with these sections:

From an Address	a feed's address, the page of a journal or a website, or a YouTube channel or its @handle; **Find**
From a Search	a PubMed or arXiv search, such as `abs:"survival analysis" AND cat:stat.ME`; **Preview** first
From Email Alerts	the alerts mailbox, and the alerts that arrived in it, ticked to follow (below)
Your Feeds	the feeds you added, with **Remove**; under **Removed Feeds**, **Restore**

For a journal's page at Wiley, Taylor & Francis, SAGE, Elsevier, Cambridge Core or Oxford Academic, **Find** gets the feed of its newest issues; for another website, the feeds its page lists, to choose from when there are several. Then pick the topic to list the feed under, or **New Topic...**.

Read

1. Click a topic or a feed on the left to see its items, or **All Feeds**. The feeds of a topic are indented under it, with a camera after the name of a feed of videos and a microphone after a podcast's. Every topic of the catalogue is listed; one with no feed subscribed starts collapsed. The last column, **Unread / Total**, counts the items of the **Items From** period. The items are shown 100 at a time, newest first; **Previous** and **Next**, above and under them, turn the pages.
2. Click an item to open it in your browser; it is then marked read, and its title is no longer bold. **Mark Unread**, shown when the pointer is on a read item, marks it unread again. An item of a podcast or a video has a **Listen** or **Watch** link; Pollis tells them by the feed, or by the item's address (a news site's video or audio page).
3. Three columns of circles, for a feed or for all the feeds of a topic: green when on, red when off, half green and half red when only some of a topic's feeds are on; the last two are under **Display**. **Subscribed**: Pollis reads the feed every hour, and a catalogue feed ticked is followed (its count is a dash until then); unticked, it is no longer read, but its items stay. **Here**: its items are shown on this page; unticked, they are hidden here and on the welcome page, but the feed is still read and archived, and the right says **No items shown** when every item of a choice is hidden. **Welcome Page**: its newest unread items are in the welcome page's **RSS** section.
4. **Items From** sets how far back items are shown: **The Last 7 Days**, **30** (the default), **90**, or **All Items Kept**. Items without a date are always shown. **On a Day...** shows the items of one day, chosen in a calendar. Only the latest items of each feed are kept here, and the archive goes back only to the day you started following it: a feed cannot tell what happened before then.
5. An item of a journal has a purple tag: the journal's level in the Norwegian register (**Level 2** leading, **Level 1** academic, **Level 0** not academic), OpenAlex's 2-year mean citedness (**Cited 7.9**), and, once you have imported them (Built-in Databases, page 40), the SJR or Impact Factor quartile and its year (**SJR Q1 2025**). The pointer on the tag shows every value known, with its source and year: SJR and its categories, SNIP, IPP, h-index, open access. Pollis knows the journal of a catalogue feed, of an email alert's article, and of a feed you added, by looking up its first items once.
6. **Journals**, next to **Items From**, shows only the items of some journals: **Academic (Level 1 or 2)**, **Leading (Level 2)**, **Open Access**, and, once imported, **SJR Q1**, **SJR Q1–Q2** and **SNIP Above 1**. Items not from a journal (news, blogs, preprints) always stay, and the counts follow the choice.

Remove, shown when the pointer is on a feed, takes it off the page; **Restore**, under **Removed Feeds** on the **Add Feed** page, brings it back.

Journal metrics

Explore > RSS > Journal Metrics, **More Metrics...** in **Journals**, the **i** next to it, or a click on a tag opens the **Journal Metrics** page: each source's metrics, their values and meaning, and more to import (page 40).

Zotero

- **Add to Zotero**, shown when the pointer is on an item, adds it to Zotero 7 or later, open on the same computer: an article from PubMed as a journal article, an arXiv, medRxiv or bioRxiv paper as a preprint, a video or a podcast episode as such, and anything else as a web page. When Zotero has collections, Pollis asks which one, starting from the last one you chose. The item then shows **In Zotero**, and is listed in the `zotero` view of PollisRSS (below), with the collection.
- For an open-access paper, an arXiv, bioRxiv or medRxiv preprint or a PLOS article, Zotero also downloads its PDF and attaches it. For any other article, use Zotero's own **Find Available PDF** (right-click the item in Zotero), which finds a free copy when there is one.
- **Export as OPML** saves the feeds you follow as one file, to follow them in Zotero (**File > New Feed > From OPML...**) or another feed reader. PubMed searches and email alerts are left out, since only Pollis reads them.

The PollisRSS database

Every item Pollis reads is added to a monthly file in `~/.pollis/rss/items`, so an item stays when its feed drops it. *PollisRSS* is a DuckDB database over those files, made with PollisDatasets the first time the Julia REPL starts (Data and Databases, page 6), and empty until feeds are read:

1. In the Databases view, click **PollisRSS**, marked **built-in**. It connects in the Julia REPL as `rss`, read-only.
2. Query its four views: `items`, one row per item (feed, topic, title, link, published, received, summary, picture and media); `feeds`, one row per feed with its number of items; `zotero`, the items added to Zotero; and `episodes`, the podcast episodes downloaded.

```
DBInterface.execute(rss, "SELECT topic, count(*) FROM items GROUP BY topic")
```

Podcasts

An item of a podcast has **Download Episode**, which saves its audio file in `~/.pollis/rss/episodes`, a folder per podcast; the notification's **Open Folder** shows it, and the item then shows **Downloaded**. The episode is listed in the `episodes` view of PollisRSS, with the file. Videos are not downloaded, since YouTube's terms do not allow it, and news pages stay links.

Email alerts

Google Scholar, JSTOR, HeinOnline, Nexis Uni, SSRN, Semantic Scholar, Web of Science, Scopus, Westlaw and Factiva send the results of a saved search only by email. Send them to an address of your own, best one used only for alerts, and Pollis turns each alert into a feed, with the title, the link, and the authors and journal of each result. Pollis reads the mailbox on your computer over IMAP, read only: nothing passes through a server.

1. **Add Feed**, **From Email Alerts**: the address, its IMAP server (suggested for Gmail, iCloud, Yahoo, Fastmail and GMX) and its password, kept in the system's keychain; then **Connect**. Gmail, iCloud and Yahoo need an *app password*; Outlook.com does not let programs in with a password.
2. Create the alerts at each source, sent to that address. The **Email Alerts** skill, in **Compose > Agent Skills**, walks you through each source and writes precise searches in its syntax, so the alerts bring few false hits.
3. **Alerts in the Mailbox** lists the alerts that have arrived, by source; the ones you tick are listed under the topic **Email Alerts**.

Google Alerts (**Show options > Deliver to > RSS feed**) and the table-of-contents alerts of many publishers are feeds already: paste their address in **From an Address**; **Search Feeds from Other Services** there shows where EBSCOhost and ProQuest give theirs.

Alerts on quotes, SEC filings and economic data

Warnings when a quote crosses a level, a company files at the SEC or a series publishes a new value are not feeds: Alerts, page 26. Press releases from PR Newswire, and what the news said around an event: Media Coverage, page 24.

If something goes wrong

Could not be read the last time; the items shown are the ones read before	The feed's site did not answer, or moved: try **Refresh** later, or remove the feed and add its new address.
No feed found at this address	On the journal's page, look for an RSS link and copy its address; for an Oxford journal, the RSS icon on its page.
The mailbox could not be read	Check the address, the server and the password; Gmail, iCloud and Yahoo need an app password, and IMAP turned on.
Zotero is not open	Open Zotero 7 or later on this computer, and click **Add to Zotero** again.
The episode could not be downloaded	The podcast's site did not answer: check the internet connection and click **Download Episode** again.
A YouTube channel shows no videos	YouTube's feeds are sometimes unavailable for a while: the items come at a later reading.

See also Data and Databases, page 6. Alerts, page 26.

Media Coverage

Explore > Media Coverage

What the news said about a company, an institution or a subject, and when.

Explore > Media Coverage counts the articles about a company, an institution or a subject over a period, by hour, day or week, marks the FOMC's releases, a company's SEC filings or a time of your own on the same chart, and lists the articles, newest first. Pollis shows the headlines and links only; an article opens in the internal browser.

Search

1. **Words**: what the articles must have. Quotes keep a phrase together (`"Apple Inc"`); `OR` between words takes any of them, in parentheses: `(iPhone OR iPad)`. Words can be a subject, such as `"going concern"`, or an institution: `"FOMC" OR "Federal Reserve"`.
2. **Sources**: tick the ones to search; Pollis keeps your choice. **Yahoo Finance** has a **Ticker** box beside it (`AAPL`): its headlines, and the company's SEC filings for **Mark**.
3. **Period**: **Last 24 Hours**, **Last 3 Days**, **Last 7 Days**, **Last 30 Days**, **Last 90 Days**, **Last Year**, or **From a Day to a Day**, with a **From** and a **To** day of your own.
4. **Mark**: **Nothing**, **FOMC Releases**, **SEC Filings of the Ticker** or **A Time**, of your own; then **Search**.

The sources

Google News	papers and sites the world over, among them those with no search of their own, such as The Washington Post and The Times; at most 100 articles a search
Yahoo Finance	the latest headlines of the ticker: the newest only, not the whole period
The New York Times	its Article Search, with your own free key: the newest 50 articles of the period
The Guardian	its Open Platform, with your own free key: the newest 1,000 articles of the period
PR Newswire	companies' and organisations' own press releases; unticked at first

The keys of The New York Times and The Guardian

Both give a free key for personal use. Beside each, **Get a Free Key** opens its sign-up page in the internal browser; **Set Key...** asks for the key, kept in the system's keychain, never in a file. **Change Key...** replaces it; left empty, the key is forgotten. A source ticked without a key says **No key set** and is skipped; the others are still searched.

Read the coverage

Coverage gives the number of articles from each source, listed of those it found, and a chart of them by hour, for up to 3 days, by day, for up to half a year, or by week. A mark is a dashed line where it falls. Click a bar to list only its articles, and **Show the Whole Period** to list them all again. **Articles** lists each with **When**, **From** (the outlet), **Headline** and **Source**. The same story told twice, by an outlet and its copy elsewhere, is listed once.

PR Newswire

A paid distribution service: companies and organisations pay it to put out their own press releases, unedited. Most are corporate (results, deals, dividends, products), but they come from every sector, from health to technology to entertainment, often before the 8-K that EDGAR shows, and with their time to the second. Its feeds list only the latest releases, so Pollis reads them every 10 minutes while **PR Newswire** is ticked here, or while there is an event study of SEC filings, and keeps them on this computer. A search finds only the releases out since then: those that have all its words, or give the ticker as the wires do, such as `(NASDAQ: AAPL)`.

From an event study

Each event of an event study has a **News** link (Event Studies, page 28): it opens Media Coverage from the day before the event to the day after, with the event marked. For an FOMC release, the words are `"FOMC" OR "Federal Reserve"`; for a filing, the company's name, with its ticker.

The other wires

GlobeNewswire and Business Wire keep programs away from their feeds, and Pollis respects that: their releases, and ACCESS Newswire's, come through Google News, with the time it gives, which can be later than the release.

What it does not do

Google News lists at most about 100 articles a search: over a long period, or for a company much in the news, it leaves gaps, and a shorter period, or more precise **Words**, finds more. FOMC releases go back about a year, as far as the Federal Reserve's feed. Pollis counts articles; it does not read them or judge their tone.

If something goes wrong

No key set: The New York Times	**Get a Free Key**, then **Set Key...** beside it, or untick it.
Not reached: Google News	Check the internet connection and **Search** again; the articles of the other sources are still shown.
Nothing found	Fewer or broader words, without quotes, or a longer period.
Search stays grey	Tick a source, and give **Words** or a **Ticker**, and a whole period.
No PR Newswire releases	Only the releases out since it was ticked are kept: search a later period.

See also RSS Feeds, page 22. Event Studies, page 28.

Alerts

Explore > Alerts · Explore > Alerts > Add Alert...

Being told when a quote or an economic series does something unusual, or when a company files something new at the SEC.

An alert watches one thing for you while Pollis is open: a *quote* at Yahoo Finance, looked up every minute; a company's *filings* at the SEC's EDGAR, looked at every 15 minutes; or an *economic series* at FRED, the ECB, the IMF or the World Bank, looked up every hour. When it fires, a notification stays until you act on it, in the colour you chose, and the status bar shows how many fired alerts you have not read. Nothing runs on a server: when Pollis is closed, nothing is watched.

An alert on a quote

1. **Explore > Alerts > Add Alert...**, with **Alert On** set to **A Quote**.
2. **Symbol**: an index (`^GSPC`), a share (`AAPL`), a future (`BZ=F`) or a currency (`EURUSD=X`); **Look Up** shows its latest value and a chart over 1 Day, 5 Days, 1 Month or 6 Months.
3. **Alert Me When It**: **Crosses Above** or **Crosses Below** a value; **Is Above** or **Is Below** it, once until it is back on the other side; or **Moves By** more than a percentage since the previous close. The value is drawn across the chart. Or one of the statistical conditions below.
4. **Shows As Information**, **Warning** or **Error**; **Fires Every Time** the condition is met anew, or **Once**, after which the alert is paused. Then **Add Alert**.

The quotes are delayed by up to about 20 minutes, depending on the exchange.

An alert on economic data

1. **Add Alert...**, with **Alert On** set to **Economic Data**, and **From**: **FRED**, **ECB**, **IMF** or **World Bank**. FRED asks for a free API key, set once with **Set FRED Key** and shared with the FRED panel.
2. **Series**: a FRED series ID (`CPIAUCSL`, `UNRATE`); an ECB dataset and key (`EXR/D.USD.EUR.SP00.A`); an IMF dataflow and key; or a country and a World Bank indicator (`PRT/NY.GDP.PCAP.KD`). **Look Up** shows its title, latest value and period, frequency, units and a chart.
3. **Alert Me When It Publishes a New Value**; crosses, or is above or below, a value; **Changes By** at least an amount from the value before; or a statistical condition.

The first look only notes the latest period; from then on, each new period is checked. A revision of a period already published does not fire. **Open Series** opens the series at its source.

Statistical conditions

For a quote or a series, an alert can fire on what is unusual for it, rather than on a value you choose. OnlineStats.jl computes these in Julia, in a hidden process that Pollis starts when needed; the first time, it installs OnlineStats and JSON3 in an environment of its own, `~/.pollis/alerts/julia`, which takes about half a minute. Your own environments are not touched. A quote's move is from one minute to the next, as a log return; a series' move is its change from the value before.

- **Moves Unusually**: a move larger than so many **Standard Deviations** of the moves before it.
- **Leaves Its Band**: the value leaves the band of so many standard deviations around its mean; once, until it is back within.
- **Moves Beyond a Quantile**: a move beyond that quantile, in per cent, of the moves before it; above 50 a rise, below 50 a fall.
- **Makes a New High or Low**: above the highest or below the lowest of so many **Values Before It**.

The mean and the standard deviation weigh recent values more: **Half-Life, in Values** is after how many a value weighs half as much. The chart then shows what the alert would have done: its band over each minute of the last five days' regular sessions, or over the series' latest values, with a dot where it would have fired, and how many times. Change the numbers until it fires as rarely as you want. **Open Notebook** beside the count opens `alert.ipynb`, which computes the same with OnlineStats.jl over the same values, to check it or try other settings; your changes to it are kept.

An alert on SEC filings

EDGAR is where companies listed in the United States publish their reports: the annual 10-K, the quarterly 10-Q, and the 8-K for events that cannot wait, such as results, a merger or a new chief executive.

1. **Explore > Alerts > Add Alert...**, with **Alert On** set to **SEC Filings**.
2. The first time, **Set Contact**: your name and email, which the SEC asks for with every request. It is kept in your system's keychain and shared with the US SEC EDGAR panel; it is sent only to the SEC.
3. **For**: **One Company**, **Several Companies**, **An Industry** or **Every Company**, the last only with **Contains**, as there are thousands of filings a day.
4. **Company**: a ticker (`AAPL`) or the SEC's CIK (320193); **Look Up** lists its recent filings, those of the chosen forms and items

in bold. **Companies**: tickers or CIKs separated by commas. **Industry**: an SIC code, such as `2834` for pharmaceuticals (**SIC Codes** opens the SEC's list); every company the SEC files under it, listed or not.

5. **Forms**: separated by commas, `10-K, 10-Q, 8-K` at first. An amendment (`10-K/A`) counts as its form; left empty, every filing fires.
6. **8-K Items**, when the forms take in 8-Ks: an 8-K then fires only when it reports one of the items ticked, such as 2.02 **Results of Operations**, 5.02 **Directors or Officers Changed** or 4.02, a restatement; none ticked, every 8-K fires.
7. **Contains**, optional: words the filing must have, in EDGAR's full-text search; an exact phrase in quotes, such as `"going concern"`. **Search** lists the filings of the last 30 days that have it, each opening the document, so you see how often the alert would fire. Then **Add Alert**.

The first look only notes the company's latest filing. From then on, each new filing of those forms fires, the oldest first; with **Once**, only the first. The notification's **Open Filing** opens the document at the SEC in your browser, and the alert keeps the second EDGAR accepted the filing. Without **Contains**, several companies and industries are looked for in EDGAR's latest filings, which go back about a day: filings made while Pollis is closed for longer are missed. With it, the full-text search goes back as far as needed, though it may index a filing some minutes after it is published.

Alert Me... in the panels

In **Explore > Access Data Libraries**, **Yahoo Finance** and **Alpha Vantage** have **Alert Me...** beside their **Symbol**, **US SEC EDGAR** beside its **CIK**, **FRED**, **ECB** and **IMF** beside each tab's **Series**, and **World Bank** on its **One Indicator** tab: it opens **Add Alert** with the symbol, company or series filled in.

Your alerts and fired alerts

Explore > Alerts lists **Your Alerts**, each with its latest value and period or latest filing, and **Pause**, **Resume** and **Remove**; a quote opens its **Quote** page, with your alerts drawn across the chart, and a filing alert opens the company's page at **EDGAR**. **Fired Alerts** lists what fired, with the value before and after, the band it left for a statistical condition, or the filing and **Open Filing**; **Mark Read** and **Mark All Read** clear them. The welcome page has an **Alerts** tab beside **RSS** with the same. Alerts are kept in `~/.pollis/alerts/alerts.json` (**Open alerts.json**), and fired alerts in `history.jsonl` beside it. With several windows open, only one of them looks things up.

Event studies

The filings an alert fires can be the events of a study of how the stock moved around them: **Start Event Study** on the alert, or **Auto Event Study** when you add it (Event Studies, page 28).

If something goes wrong

Yahoo Finance does not know . . . , or could not be reached	Check the symbol at Yahoo Finance's site, as it is written there (`^GSPC`, `BZ=F`), and the internet connection.
The SEC does not know . . . , or could not be reached	Try the CIK instead of the ticker: some filers have no ticker in the SEC's list. Check the internet connection.
Needs your SEC contact	**Set Contact** on the Add Alert page, under **SEC Filings**: a name and an email address.
Needs your FRED key	**Set FRED Key** on the Add Alert page, under **Economic Data**.
Needs Julia	A statistical condition is computed in Julia: install it (Install Julia, page 2), and the alert goes on.
Julia is installing OnlineStats.jl	The first time only; the alert is checked again within the minute.
An alert fired late	Quotes are delayed by up to about 20 minutes, filings are looked at every 15 minutes and series every hour; nothing is looked up while Pollis is closed.

See also RSS Feeds, page 22. Data and Databases, page 6.

Event Studies

Compose > Workflows > Event Studies...

How stocks move around SEC filings, and rates, yields and stocks around the FOMC's releases, a minute at a time; and how an outcome moves when units are treated at different times.

An event study measures how something moves around events. A *market event study* follows prices in the minutes around them: the *SEC filings* an alert fires, with the minute prices of each filer's stock and of a benchmark; or the *FOMC's releases*, with federal funds and SOFR futures, Treasury yields, TIPS and the S&P 500. While Pollis is open it captures the prices from Yahoo Finance as they come and draws the result on the page. A *panel event study* follows an outcome over the years, quarters or months after units are treated at different times: states adopting a policy, firms a rule, or the clinics of a stepped-wedge trial crossing from control to treatment one group after another, from a table of your own. Either way the study is a folder that a notebook analyses with PollisEventStudies.jl.

A study of SEC filings

1. An alert on SEC filings chooses the events: its companies, forms, 8-K items and words (Alerts, page 26). **Add Alert...** on the page makes one.
2. **Compose > Workflows > Event Studies...**, **New Study**: a **Title**, **Kind Market Event Study**, **Events SEC Filings**, and the alert.
3. **Benchmark**: a Yahoo Finance symbol, `^GSPC` for the S&P 500 at first. **Normal Return**: **Market-Adjusted** takes the benchmark's return out; **Market Model** takes out alpha plus beta times it, estimated on the minute returns of the regular sessions of the **Estimation Days** before the period.
4. **Event Time**: **First Public**, the earlier of the company's newswire release and EDGAR's acceptance, as results are often on the wires hours before the 8-K; or **EDGAR Acceptance** alone.
5. **Periods**: minutes **Before** and **After** the event, at first 30 and 120; **Next Open** also follows an event outside the regular session from the next open. **Add Period** adds another. Then **Make Study**.

A new study starts with the filings the alert would have fired in the last 30 days, as Yahoo Finance keeps minute prices for 30 days, and adds each filing the alert fires from then on. On the alert, **Start Event Study** opens this page with the alert chosen, and **Auto Event Study** starts a study with the settings above the first time the alert fires.

The newswire release

For each filing, Pollis looks for the company's release on GlobeNewswire, Business Wire, PR Newswire or ACCESS Newswire from a day before EDGAR accepted it to a day after: in Google News, and in PR Newswire's own feeds, which Pollis reads every 10 minutes while there is a study of filings and which give the time to the second (Media Coverage, page 24). The events show the **Release**, the **Accepted** time and the **Lead** of one over the other; **News** opens Media Coverage around the event.

A study of FOMC releases

New Study, **Market Event Study**, **FOMC Releases**: which releases are events (the statement, the Summary of Economic Projections, the minutes, other), and the **Instruments**, by what their move tells: **Target Rate Surprise** (FF1, the current month's federal funds future), **Path and Forward Guidance** (FF2 to FF4, SOFR futures, the 2-year note future ZT) and **Fed Information Effect** (`^FVX`, `^TNX`, TIP, `^GSPC`, ES). The period is **10 Min Before, 20 After**, the tight window of Gürkaynak, Sack and Swanson (2005). A new study starts with the releases of the last 59 days, with five-minute prices for those older than 30 days.

Read a study

Your Studies lists each study, with its events and how many are still capturing prices. Open one:

- A study of filings draws the cumulative average abnormal return (CAAR), in per cent, with its 95% band, and each event's in grey, **From the Event** or **From the Next Open**; under it, the articles about the events. **Events** lists each filing: **Company**, **Form**, **When** (during the session, before the open or after the close), **Prices** and its **CAR**.
- A study of FOMC releases draws each instrument's average move from the start of the period, with its 95% band, in basis points for rates and yields and in per cent for prices, and lists the releases.
- **Open Folder**, **Edit study.toml** and **Open Notebook**.

The folder and the notebook

Each study is a folder in `~/.pollis/event-studies`: `study.toml`, its settings; `events.csv`, the events with their times in UTC; `prices/`, a file of bars per event; and, for filings, `news/`, the articles about each. **Open Notebook** opens `study.ipynb`, which installs PollisEventStudies.jl from the Pollis registry and runs it on the folder:

- filings: for each period, the average and cumulative average abnormal returns a minute at a time with their t, the CAR's t, the Boehmer, Musumeci and Poulsen (1991) test, and abnormal volume;
- FOMC releases: each instrument's move at each release, the mean move with its t, and the mean move a minute at a time.

The results are written to `results/`. Events still capturing prices count up to the last complete minute: run the notebook again as they come in.

What it does not do

Prices are captured only while Pollis is open; a study catches up on reopening within Yahoo Finance's 30 days of minute prices (60 of five-minute prices), and an event older than that has none. A filer without a ticker has no prices. An expired futures contract is gone from Yahoo Finance, so FF1 of a past month has none, and Yahoo Finance has no 2-year yield in minutes: ZT stands for it, as a price.

If something goes wrong

No alert on SEC filings yet	**Add Alert...** on the page, with **Alert On** set to **SEC Filings**.
No prices at Yahoo Finance	The stock did not trade around the event, or Yahoo Finance does not list it.
Older than the 30 days of minute prices	The event came before what Yahoo Finance keeps: it stays in the events, without prices.
Some are not among the events	The last 30 days had more filings than EDGAR's full-text search reads: narrow the alert.

A panel event study

A panel study is made from a table of your own rather than from prices Pollis captures, and nothing happens until its notebook runs.

1. **New Study**: a **Title** and **Kind Panel Event Study**.
2. **Table**, **Choose File...**: a CSV file with a row per unit and time (or per individual, unit and time). Its columns and first rows are shown, and it is copied into the study's folder as `panel.csv`. Times are whole numbers: years, or quarters or months numbered in order.
3. **Columns**: the **Unit** (also the clusters of the standard errors), the **Time** and the **Outcome**. **Treated**: a **Treatment Indicator**, not 0 from the time a unit is treated, or a **First Treated** column, the time each unit is first treated, missing or 0 for never. Choose each one: what a column means cannot be read from its name, and a column called `treat` may only say which units are ever treated.
4. **Estimators**, one or more:
   - **Two-Way Fixed Effects and Sun and Abraham**: the regression on leads and lags of the first treatment with unit and time effects, and Sun and Abraham's (2021) estimator, which stays right when the effects differ between cohorts;
   - **Callaway and Sant'Anna**: their (2021) effect of each cohort at each time, against the **Control** units (**Never Treated**, **Not Yet Treated**, **Treated Last**, or **Auto**, the never treated when there are any), by a **Method** (**Doubly Robust**, **Inverse Probability Weighting**, **Outcome Regression**), and its averages;
   - **Stepped Wedge**: the mixed model of Hussey and Hughes (2007) for a trial whose clusters cross to treatment one group after another, with an **Effect** (**Immediate Treatment**, one effect; **Exposure Time Indicators**, one for each time since the cross; or **Heterogeneity by Exposure Time**), averaged over exposure times **From** to **To** or taken **At** one, and an **Outcome Type**, continuous or binary (0 or 1, or successes out of **Trials**). It needs a treatment indicator.
5. **Event Times**: the **Leads** before the first treatment, for the test of pre-trends, and the **Lags** after it. Then **Make Study**.

Run and read it

Your Studies shows the table, its columns and the estimators. **Open Notebook** opens `study.ipynb`, which installs PollisEventStudies.jl from the Pollis registry (and MixedModels.jl for the stepped wedge), runs the estimators `study.toml` names on `panel.csv` and writes the results to `results/`. The page then draws:

- each estimator's effects by event time with their 95% intervals, 0 being the first time treated (the stepped wedge's exposure time 1), and a table of them;
- the tests of pre-trends: an F test of the effects before treatment;
- Callaway and Sant'Anna's averages: simple, over event times, over cohorts and over calendar times;
- the stepped wedge's estimate, with the intracluster correlation.

The page redraws when `results/` changes: after an edit of `study.toml`, run the notebook again.

Which estimator

Two-way fixed effects averages the effects with weights that can be negative when the effects grow over time, because units treated earlier serve as controls for those treated later. Sun and Abraham, and Callaway and Sant'Anna, compare each cohort only with units not yet or never treated; when they agree with each other and not with two-way fixed effects, trust them. Units treated from the first time have no time before treatment to compare with, and the estimators leave them out.

If something goes wrong

No results yet	A panel study: open its notebook and run it.
Pre-trends: not tested	Callaway and Sant'Anna's test takes every cohort's effect at every time before treatment; with more of them than units, as with many cohorts over many years, it cannot be made. Their effects by event time before 0 still show any pre-trend.
Not a table	The file needs a header row and three columns or more, separated by commas.
The stepped wedge needs a treatment indicator	Under **Treated**, choose **Treatment Indicator** and a column that is not 0 from the time a unit is treated.

See also Alerts, page 26. Media Coverage, page 24. Replication, Methods and Workflows, page 13.

Collaboration

Compose > Collaboration

Working with others at their own computers: a chat, and sharing code, datasets, files and databases.

A session connects up to five people, each at their own copy of Pollis: two colleagues, a student and a tutor, or a small team. Everyone keeps their own editors and notebooks; in the chat you talk, and you share what you choose, when you choose. Nobody can change anything on anyone else's computer.

What you need

- Pollis on each computer, and an Internet connection.
- A network that lets sessions through. To check, choose **Compose > Collaboration > Test Connection...** before the session, and send its results (**Copy Results**) if you need help.

Start a session

1. Choose **Compose > Collaboration > Start Side by Side Session...**
2. Type your name, as the others will see it. The chat opens beside your editor, and the invitation is copied to the clipboard.
3. Paste the invitation into an email or a message to the others. To copy it again, use **Copy Invitation** at the top of the chat, or **Compose > Collaboration > Invite People...**

You are the *host*: you admit the others, up to five in all, and when you leave the session ends for everyone.

Join a session

1. Choose **Compose > Collaboration > Join Session...**
2. Paste the invitation you received, or type its code: four groups of five letters and digits, such as `7KQ2M-....`
3. Type your name. The chat opens and says when you have joined.

The chat

Write a message and press **Enter**; **Shift+Enter** starts a new line. Paste or drag an image into the message: larger images are reduced to fit the limit of 2 MB. The **+** button in the message box adds:

- **Image**: a PNG, JPEG, GIF or WebP image, shown in the chat.
- **File**: any file up to 5 MB. The others get a card with **Save As...**; Pollis never opens or runs it. Dragging a file that is not an image into the chat shares it the same way.
- **Dataset**: a CSV file up to 5 MB (see the next page).
- **Database**: a DuckDB database up to 5 MB (see the next page).

Show Participants lists who is in the session and whether each connection is direct or through a relay.

> **The invitation is the key**
>
> Anyone with the invitation can join while the session lasts, and receives whatever is shared from then on. Send it only to the people you mean, and do not share personal or confidential data in a session whose invitation may have gone further.

Share your code

While you are in a session, a bar at the bottom of each editor and notebook shows **Share My Editor**. It sends a snapshot of what you see, unsaved changes included; a notebook is sent without its outputs. For a Pluto notebook in Pollis's browser, use the **Share My Editor** button in the browser's toolbar. Sharing again replaces the earlier snapshot. Code files and notebooks up to 2 MB can be shared.

The others see your snapshot read-only, with two buttons below it:

- **Compare with Mine**: your snapshot and their own editor side by side, line by line.
- **Copy to My Editor**: replaces the content of their own editor, or the cells of their own notebook, with yours. **Undo** brings theirs back, except in Pluto.

Share data

- **A dataset** (CSV): **+ > Dataset** in the chat, **Share Dataset** on a CSV file in the Explorer, or **Share Dataset** in the toolbar of a Julia table viewer. Pollis asks before sending. The others get a card with the first five rows and **Preview**, **Save to My Workspace** and **Load into My Julia Session**, which reads it into a Julia table named after the file.
- **A database** (DuckDB): **+ > Database**. Pollis asks before sending the whole database. The others get a card with **Save As...** and **Attach in My Julia Session**, which opens it *read-only*, as a variable named after the file, and shows its tables in the Databases view.

> **Before sharing a database**
>
> Changes made in a Julia session that still has the database open may not be in the file yet: close that connection, or run `CHECKPOINT`, before sharing. And attach only databases from people you trust: a database can hold views that read files on the computer where they are queried.

Leave or end the session

Leave Session, at the top of the chat or in **Compose > Collaboration**, leaves the session. When the host leaves, the session ends for everyone. If the host's connection drops, the others wait a minute for it to come back. After the session the chat stays open, to be read, until you close it.

How you are connected

Free public servers introduce the participants to each other; after that each pair connects directly. The servers never see the session code or anything you share. By default they are nostr relays; on networks that block those, set `pollis.collaboration.signalling` to `mqtt` before starting the session (the invitation tells the others). A relay for networks that block direct connections is not set up yet.

If something goes wrong

"Could not reach the session"	Check the code, and that the host is still in the session. Then run **Test Connection...** on both computers.
Test Connection: no introduction server reachable	The network blocks Pollis (often a firewall or proxy): try another network, such as a phone's hotspot.
Test Connection: nostr blocked, MQTT reachable	The host sets `pollis.collaboration.signalling` to `mqtt` and starts a new session.
Two people never connect	Both networks may block direct connections, which need the relay that is not set up yet: one of them tries another network.
"The session is full"	A session has at most five participants, the host included.
A file, dataset or database is refused	It is over 5 MB, or not what it says it is: send it the usual way.

> **Not available yet**
>
> **Start Same Page Session...** (editing together), **Invite an Agent...**, **Session Settings...**, **Classroom** and **Simulation Games** are in the menu but not built yet: they say so when chosen.

See also Data and Databases, page 6. Inside a Panel, page 4.

Orchestration

Compose > Orchestration · Compose > Coding Agents

Choosing and starting coding agents, and routing tasks to models.

Some tasks of the **Compose** menu are done by a *coding agent*, a program that writes code and documents in a terminal: creating panels, packaging toolboxes, writing methods and vignettes, polishing reports. Their menu items carry a sparkle, because they run on your agent's account and cost tokens. Pollis works with any agent: you choose which, and which model each task runs on.

Choose your coding agent

1. Choose **Compose > Coding Agents**. The page lists Claude Code, Codex and OpenCode, each with its command and whether it is **On This Computer**: **Agent Installed**, or **Install Agent**, which opens its install page.
2. Click **Use** beside yours; it then shows **In Use**.
3. Click **Install Skills**, so that the agent knows the Pollis skills in any folder, even outside Pollis. Later, **Update** installs their latest version.

Each agent also has a *local* command, under **Local and Cloud Models**, that runs it on a model on this computer through Ollama or LM Studio: free and private. For any **Other Agent**, type its command line with `{prompt}` where the first message goes; without it, Pollis copies the message to the clipboard, to paste into the agent. The choice is kept in `pollis.agents.command`.

What happens when a task starts

Pollis opens a terminal, starts the agent there and gives it a first message that points it to the skill for the task. From then on you talk to the agent in that terminal.

Agent Skills

A *skill* is the know-how an agent reads to do a task. **Compose > Agent Skills** lists them, downloaded in their latest version when the page opens: create-panel, package-toolbox, prepare-method, references, write-method and write-vignette, each with its **Tasks** and **Example Requests**. **View Skill** opens it; **Run with My Agent** starts your agent on it.

Model Routing

Not every task needs the strongest model. **Compose > Orchestration > Model Routing** has three *tiers*:

- **Light**: small tasks, on a fast, cheap or local model.
- **Standard**: everyday tasks.
- **Strong**: hard tasks, where a mistake costs most.

Give each tier the command line of an agent, with `{prompt}`, such as `claude -model haiku {prompt}` for Light. An empty tier runs your main agent, and so does a tier whose program is not installed. Each task runs on a tier by default:

Task	Menu	Tier
Polish with Agent	Reports	Light
Write a Vignette, Prepare for Registration	Methods	Standard
Package a Toolbox	New Panels and Toolboxes	Standard
Tables from the Article, Write a Method	Methods	Strong
Create a Panel, Create Panels from a Request	New Panels and Toolboxes	Strong

In **Tasks**, choose another tier for a task, or **Main Agent**. The settings are `pollis.agents.tiers` and `pollis.agents.taskTiers`.

Local Models

Compose > Local Models has **Hugging Face Models** and **Kaggle Models**, to find models, and **Local Model Runners**. A local model runs only through a *runner*, a program that loads it and serves it on this computer: Ollama, LM Studio, llama.cpp or Apple MLX. The page shows which are installed, and how each agent reaches each one.

> **An agent works on your files**
>
> A coding agent reads and writes files, and runs commands, in its terminal. Read what it proposes, and check its results, as you would a colleague's.

If something goes wrong

Choose the coding agent Pollis starts	Choose one in **Compose > Coding Agents**, then the task again.
The coding agent … was not found	Install it (**Open Install Page**), or choose another one.
… tier: … isn't installed	The task ran on the main agent: install the tier's program, or change its command.

Not available yet

Writer and Checker..., **Two Independent Transcriptions...** and **Race...** are in **Orchestration** but not built yet.

See also Toolboxes, page 10. Methods, page 16. Reports, page 20.

Literature Radar

Compose > Literature Radar

Finding the articles that matter on a topic, bringing in references of your own, and following citations from a few works you know.

Literature Radar asks two free, open indexes of scholarly works, OpenAlex and Semantic Scholar, and keeps what you choose in PollisReferences (Built-in Databases, page 40). It has four pages: **Discover Literature...**, **Import References...**, **Browse References...** and the **Snowball Search Map**.

Discover Literature

1. Under **Indexes**, tick OpenAlex, Semantic Scholar or both. Each lists its 200 most cited articles; with both, the two lists are merged into one.
2. Type a **Topic**, and choose **Search In** (**Title and Abstract**, or **Title**), a **Field**, the **Years**, **Citations At Least** and **Recent Citations In the Last** N **Years**.
3. To keep only the best, choose an **Influence** percentile, a **Journal Ranking** (2-year mean citedness, the Norwegian levels, or SJR quartiles once imported), or **Only Articles Anyone Can Read**.
4. Click **Search**. The table shows the citations **Since Publishing** and in the **Last N Years**, **Influential** citations, **FWCI** and **Percentile**; versions of one article are shown once, with **+N versions**. Click a title for its **Abstract**.

Tick articles, then act on them: **Save to PollisReferences**, **Copy BibTeX**, **Open PDFs**, **Show in Panel...**, **See in References**, or **Snowball Search...**, which makes them the seeds of a map. **Reconcile Sources** asks each index for the counts of the articles only the other found.

Keys are optional and free

Without a key, OpenAlex gives everyone on your network about 100 searches a day between them, and Semantic Scholar asks searches to wait at busy times. A key of your own has its own budget or rate. Click **Get a Free Key**, then **Set Key...**; Pollis keeps it in the system's keychain.

Import References

Bring references you already have: **Paste** them, or choose **Files** and **Choose Files...**. BibTeX, BibLaTeX, RIS, CSL-JSON and lists of DOIs are read; the format is told by the text itself. Tick **Look Up in OpenAlex and Semantic Scholar** for their citation counts and the fields your file lacks (by DOI, else by title), then **Import**. The **Tick** links (**All**, **None**, **Found**, **Not Found**, **Not Looked Up**, **Not Yet Saved**) choose what **Look Up Ticked**, **Remove Unticked...** and **Save to PollisReferences** work on.

Browse References

Browse References... lists everything in PollisReferences: the works Pollis cites, those you saved and those you imported, with the same actions. It is where a Snowball Search Map finds its seeds when you type in **Add Seeds**.

Snowball Search Map

From a few works you know, follow the works citing them, a step at a time, until the steps bring little new. The map has two pages of its own (Snowball Search Map, page 38).

Choosing what to keep

Discover Literature lists the most cited, which favours older work. Sort by **Last N Years** or **FWCI** (citations against works of the same field and year) for recent work that already stands out, and use **Percentile** to compare fields. **Influential** counts the citations Semantic Scholar finds weighed on the work, not only mentioned.

If something goes wrong

The free budget of OpenAlex … is used up for today	Try again tomorrow, or set a free key of your own.
OpenAlex could not be reached, or refused the search	Check the connection, and try again.
Semantic Scholar could not be reached, or is too busy just now	Search again a little later, with OpenAlex alone, or with a key.
… has no record of this work	The other index may have it: look up its DOI there.
Not found in OpenAlex (Import)	Its title or first author differs from OpenAlex's: correct the file, or keep it as it is.

See also Inside a Panel, page 4. Snowball Search Map, page 38. Built-in Databases, page 40.

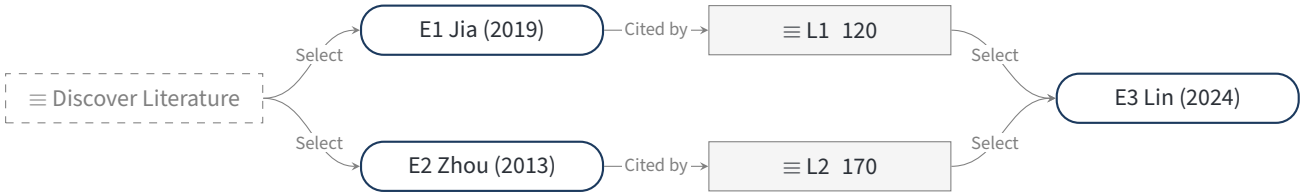
Snowball Search Map

Compose > Literature Radar > Snowball Search Map

Following citations from a few works you know, a step at a time, and telling when to stop.

A keyword search finds what uses your words. A *snowball search* finds what a literature actually builds on: start from a few works you know are central, the *seeds*, list the works citing them, make the best of those seeds in turn, and go on. Works cited by several seeds rise to the top, whatever words they use. Pollis follows citations *forward*, to the works citing each seed, and *backward*, to the works each seed cites, from OpenAlex.

Reading the map



- A dashed box is where seeds came from: **Discover Literature**, or **PollisReferences** for those found in **Add Seeds**.
- A pill is a seed, filled when chosen: **E1 Jia (2019)** is the first seed, by Jia, of 2019. Seeds are numbered E1, E2… in the order they were added.
- A box with ≡ is a list: **L1 120** is the first list, the 120 works OpenAlex has citing E1 (up to the 2,000 most cited). Click it to see only its works in the pool.
- **Select** arrows show where a seed was chosen from; **Cited by**, **Cites** and **Related** arrows, the list a seed gave. Above, E3 was chosen from the pool, where it appears in both L1 and L2: it cites both seeds.

Under the map, **Layout: Landscape** grows the map to the right, and **Portrait** grows it down, which shows more of a long search at once.

The pool

Every list pours into one **Pool**, each work once, ranked by **Links**, the number of seeds it cites or is cited by, then by **Cited**. A work that cites three of your seeds is more likely about your question than one that cites one, however cited. **From** names the lists a work is in, **Show** narrows the pool to **All**, one list, **New Since Last Step**, or **Candidates**, the works of the top 20 neither seeds nor excluded; **Seed** beside a work makes it a seed. Click **Links**, **Year** or **Cited** to sort by it, largest first; click again for smallest first. Sorted by **Links**, works tied on seeds come in order of their influential citations, the filled dots.

Related lists, for each seed, the works Semantic Scholar recommends by the likeness of their titles and abstracts, mostly recent, up to 500; for a seed it has no record of, often an older one, the ten or so OpenAlex relates to it. They join the pool marked ≈, but a recommendation is not a citation: **Links** does not count them.

A step at a time

1. Add two to five seeds you trust: too few and the pool follows one author's network; too many and every step costs many requests.
2. Click **Cited By**, for the works citing each seed, or **Cites**, for the works each seed cites. Each seed not followed so yet gives a list; a seed already followed is skipped.
3. Read the top of the pool, and make seeds of the works that belong to your question, especially recent ones with several **Links**.
4. Click **Cited By** or **Cites** again, and repeat.

Clicking a seed on the map, or its E number in **Seeds**, *chooses* it: it is drawn filled, a note under the map names the seeds chosen, and **Cited By**, **Cites** and **Related** follow those alone. Click a chosen seed again to drop it, or **All Seeds** to follow all seeds again. The **×** beside a seed removes it: it and its lists stay on the map, dashed, and **Links** no longer counts it.

When to stop

The line above the pool reads, for instance, **286 works from L1–L2 · the last step added 166, 9 of them in the top 20**. While a step still brings works into the top 20, the literature is still opening up. When a step adds few or none there, the search has *converged*: the central works are in the pool, and further steps bring only the periphery.

The search goes in *rounds*: a round is the works made seeds, and the **Cited By** or **Cites** lists that follow them. After each round, the line below it compares the pool's top 20 with the round's before, for instance **Round 3 (L5–L6, from E4, E5): 4 works new to the top 20**, and marks those works ↑ **New in the top 20**. Make the best of them seeds and follow them. When a round brings nothing new into the top 20, the line reads **The search has saturated**: the stopping point the guidelines for systematic reviews call *saturation*. **Related** lists change no work's **Links**, so they count in no round.

Run to Saturation runs the rounds for you. It follows the seeds not followed yet; then, each round, it makes the best candidates seeds and follows them, until a round brings nothing new into the top 20. Beside it, **Seeds a round** (5 unless you change it) sets how many candidates a round makes seeds, **Rounds at most** (6 unless you change it) when to stop if the search has not saturated, and **Following** whether by **Cited By**, **Cites** or both. Pollis remembers what you set. **Stop** ends the run once its round ends. The top ranks works by how central they are, not by whether they are on your question: **Exclude** beside a work keeps it out of the top, and so out of the candidates, and **Include** ranks it again.

How they cite it

Tick **Ask Semantic Scholar About the Citations** before **Cited By** or **Cites**. Each dot under **Links** is then a seed the work cites, or is cited by: filled when Semantic Scholar finds the citation *influential*, hollow otherwise. Click the dots for, seed by seed, why one cites the other (**Background**, **Method** or **Result**) and the sentences citing it. A work cited only in passing in the introduction is rarely a good seed; one that uses a seed's **Method** often is. **No Semantic Scholar record** leaves the ranking as it is.

Saving a map

Each step is kept in the map's record. **Save...** writes it to `~/.pollis/literature`: a TOML file of the name you give, with the seeds and steps, and a DuckDB file beside it with the works found. **Open...** brings a saved map back; **New Search** starts again.

If something goes wrong

OpenAlex has no record of E1 that Pollis could find	Its title or first author differ from OpenAlex's: add it from PollisReferences with its DOI.
The works citing E1 are L1 already	Each seed is followed once: choose another seed, or make new ones.
The free budget of OpenAlex … is used up for today	Try again tomorrow, or set a key in Discover Literature.
Semantic Scholar could not be reached, or is too busy just now	Click the dots again later.

See also Literature Radar, page 36. Built-in Databases, page 40.

Built-in Databases

Databases view > PollisDatasets, PollisRSS, PollisJournals, PollisReferences

The four databases that come with Pollis: what each holds, and how to add journal metrics.

Pollis makes four DuckDB databases the first time the Julia REPL starts (Getting Started, page 2), and keeps them in `~/.pollis/databases`. All four are read-only: Pollis rebuilds them when what they come from changes, and your own tables go in a database of your own. In the Databases view each is marked **built-in**; clicking it connects it in the Julia REPL. The `duckdb>` terminal, the `sql>` mode and `PollisDB.connect()` attach all four (Data and Databases, page 6).

Database	In Julia	What it holds
PollisDatasets	`pollis`	The 57 datasets of PollisDatasets, a table each, with their descriptions.
PollisRSS	`rss`	Every item of the feeds you follow, kept by month; empty until feeds are read (RSS Feeds, page 22).
PollisJournals	`journals`	About 250,000 journals, conferences and book series, from OpenAlex; the Norwegian register's levels; and the journal metrics you import.
PollisReferences	`references`	Every work Pollis cites, and those you save from Literature Radar (page 36) or add to a panel's **Explore References**, with abstracts, keywords and dated citation counts.

PollisRSS

`items`	One row per item: feed, topic, title, link, published, received, summary, picture and media.
`feeds`	One row per feed, with its topic and number of items.
`zotero`	The items added to Zotero, with the collection.
`episodes`	The podcast episodes downloaded, with the file.

PollisJournals

`journals`	One row per journal, conference or book series in OpenAlex: title, ISSNs, publisher, country, field, works, citations, 2-year mean citedness, h-index, open access and article charges.
`journal_years`	The works and citations of each journal, by year.
`journal_issns`	Every ISSN, print and electronic, with its journal's `id`.
`norwegian_register`	The 40,000 journals and series of the Norwegian register, with field, DOAJ and the `journal_id` of the journal in `journals`.
`norwegian_levels`	Their level each year since 2004: 0 not academic, 1 academic, 2 leading, X under review.
`metrics`	The journal metrics you import (below): title, category, ISSNs, year, source, then the metrics.
`journal_metrics`	The same, with the `journal_id` of the journal in `journals`, found by ISSN.
`articles`	The articles of email alerts and those saved from Literature Radar, with journal, citations, FWCI and percentile.

OpenAlex's data are CC0, free for any use. The Norwegian register is under the Norwegian Licence for Open Government Data (NLOD) 2.0, distributed by HK-dir. PollisJournals.jl brings a new copy of both every month, and Pollis rebuilds the database when it is installed. `metrics`, `journal_metrics` and `articles` are views over your own files in `~/.pollis/journals`: always current, and never replaced by an update.

PollisReferences

`all_references`	Every work once, with its key (Appendix: Reference Keys), type, authors, venue, DOI, abstract and keywords; `origin` is `pollis`, `corrected` (yours, over Pollis's) or `user`.
`all_counts`	Citation counts, each with its source and the date it was observed, never added up.
`all_versions`	The records of a work you saved in OpenAlex and Semantic Scholar: the article, a preprint, a working paper.
`user_panels`	The works you added to, or hid from, a panel's **Explore References**.

Pollis's own (`pollis_references`, `pollis_counts`) come from PollisReferences.jl and are replaced at each update. Yours (`user_references`, `user_counts`, `user_versions`) are views over your files in `~/.pollis/references`, never replaced. A work Pollis already has is not added again: it keeps Pollis's key, and only its versions and counts are yours.

Import journal metrics

Impact factors, quartiles and ratings are not free to share, so PollisJournals does not come with them: you bring your own, downloaded from their source, and Pollis adds them to the database. The **Journal Metrics** page (**Explore > RSS > Journal Metrics**, or right-click **PollisJournals**) has a card per source, with its download page, what you have imported and **Remove**; **Ask an Agent** starts your coding agent on the Journal Metrics skill, to help you choose, download and import.

Source	Metrics	Where to get it
SCImago Journal Rank	SJR, its quartile, h-index	Free: `scimagojr.com`, **Download data**, a year at a time
CWTS Journal Indicators	SNIP, IPP	Free: `journalindicators.com`, one file with every year
ABDC Journal Quality List	A*, A, B, C (business and economics)	Free: `abdc.edu.au`, the Excel file of the latest list
Scopus	CiteScore, its quartile	Free with an Elsevier account: `scopus.com/sources`
Journal Citation Reports	JIF, 5-year JIF, JIF quartile, JCI	With a Clarivate subscription, often through a university library

1. Download the file, as CSV or Excel.
2. On the **Journal Metrics** page, **Import File...**; or right-click **PollisJournals** and choose **Import Journal Metrics...**. Choose the file.
3. Pollis tells the source by the file's columns, so you do not have to. The year comes from a column of the file, its headers or its name; when none has it, Pollis asks.
4. Refresh PollisJournals: the rows are in `metrics` and `journal_metrics`.

Each file imported becomes a file of Pollis's own in `~/.pollis/journals/metrics`, which the two views read; the file you downloaded is no longer needed. Importing the same file again replaces its rows, and the metrics stay when Pollis or PollisJournals is updated. On the RSS Feeds page, the tags of journal items then show the SJR or Impact Factor quartile, and **Journals** offers **SJR Q1**, **SJR Q1–Q2** and **SNIP Above 1**.

Query

A journal's metrics from every source, newest first:

```
SELECT year, source, sjr_quartile, snip, jif FROM journal_metrics
        WHERE title = 'Econometrica' ORDER BY year DESC
```

The works you saved from Literature Radar, with their latest OpenAlex count:

```
SELECT r.key, r.title, max_by(c.count, c.date) AS cited
FROM PollisReferences.all_references r JOIN PollisReferences.all_counts c USING (key)
        WHERE r.origin = 'user' AND c.source = 'openalex' GROUP BY ALL
```

If something goes wrong

Import Journal Metrics... is not in the menu	Right-click **PollisJournals** itself, not one of its tables, or use the Command Palette.
Not a file Pollis can import	It reads the exports of the five sources above only. A file of an older edition may have other column names: **Help > Report Issue**.
A row has no `journal_id`	No journal in OpenAlex has its ISSN, as for many book series and conference proceedings. Its metrics are still in the views.
A journal has two rows for a year	One per source; SCImago also lists some journals twice, under an old and a new title.

See also Data and Databases, page 6. RSS Feeds, page 22.

Appendix: Reference Keys

pollis.bib · Compose > Reports

How every work cited in Pollis is kept once, and how to write its key from the work in front of you.

Pollis keeps rich references: besides its title, authors and venue, each work has, wherever they can be found, its abstract, its keywords and its citation counts over time, each marked with where it came from. Every article, book, report and package referenced in Pollis, by a panel, a notebook or a method, is kept once, in PollisReferences.jl, a package of its own. Panels, notebooks and methods name a work by its key only; the `pollis.bib` that reports cite from, and the references a panel shows, are built from it. A correction is therefore made in one place.

The key

A key is the authors, the year and a letter: `FF1993a` (Fama and French, 1993), `Bre2001a` (Breiman, 2001). Anyone with the work in front of them can write everything up to the year; the letter is the database's.

Authors	The key starts with	Example
One	The first three letters of the surname: a capital, then lower case. A shorter surname is used whole.	`Bre`, `Bol`, `Wu`
Two to five	The initial of every surname, in the order of the work, not alphabetical.	`FF`, `TL`
More than five	The first five initials, then `...`	`ABCDE...2019a`
Some not found, after looking	The initials of those known, then `...`; one known author keeps the three letters. Never a shortcut: five known authors are five initials.	`AB...1975a`, `Rac...2020a`
An organisation	The capitals of its name as printed, without Inc. or Ltd.; a one-word name that is not an acronym reads like a sole author.	`ECB`, `IMF`, `AV`, `EI`, `Kag`
None	`U`, for unknown.	`U2026a`

Surnames

- **Always the last word of the surname:** Lopez de Prado → `Pra`; Puigdomènech Badia → `B`; van Gogh and Van Gogh → `G`; El Ghaoui → `G`.
- **A hyphenated surname is one word,** read from its start: El-Ghaoui → `E`; Rhys-Jones → `R`; Ben-Tal → `B`.
- **As the work prints it.** Accents are removed (Sánchez → `San`, Øksendal → `Oks`), and an apostrophe keeps the first letter (O'Neill → `O`).

The year and the letter

- **The year** has four digits: `1924`, `2024`. A forthcoming work takes the year expected; an undated one, `0000`.
- **The letter** is always there: `a`, then `b`, … `z`, then `aa`, `ab`, …, in the order the works are added. If `FF1999a` and `FF1999b` are taken, the next work of Fama and French from 1999 is `FF1999c`.
- **A key is given once** and never changes, even if the work's authors or year are later corrected.

Add a work

1. Look the work up by its DOI and its title: it may be there already, perhaps under another spelling of a name (El Ghaoui, El-Ghaoui).
2. Write the key up to the year, from the work itself.
3. The database adds the letter, and lists the works already under the same authors and year.

A package is cited by the paper it asks to be cited by, in its `CITATION.bib` or `CITATION.cff`; when there is none, by the package itself, with the authors it names.

What the database keeps

`references`	One row per work: its type (article, book, inproceedings, incollection, techreport, phdthesis, misc), title, authors, year, venue, volume, pages, DOI, arXiv and OpenAlex ids, links, a free full text when there is one, the abstract and the keywords, each with its source. A row is changed only to add what is missing or to correct a mistake.
`counts`	The citations of a work as a source reported them on a day: key, count, source and date. Rows are only added, so each source's counts can be followed over time.

The type of a work is stated where it is added, never guessed from its venue. Abstracts come from the DOI's record, OpenAlex, Semantic Scholar or arXiv, and a package's from its About text on GitHub. Keywords are OpenAlex's, which it assigns itself, arXiv's subject categories or a package's GitHub topics. Citation counts from OpenAlex and Crossref are added every month; Google Scholar has no interface for this, so its counts are entered by hand.

If something goes wrong

The key you wrote is taken	Expected: the database gives the next letter. Check the works it lists under that key first: one may be the same work.
A work is in twice	Its name was spelt two ways. Keep the older key, and correct the other's uses.

See also Reports, page 20. Built-in Databases, page 40.